

CS Study

Database - Join

목차

1

Join 개념과 종류

2

USING vs ON

3

WHERE vs ON

4

Join과 성능

5

Join 알고리즘

목차

1

Join 개념과 종류

2

USING vs ON

3

WHERE vs ON

4

Join과 성능

5

Join 알고리즘

Join이란?

RDB 중복 데이터 피하기 위해 **정규화** 진행

분리된 테이블 다시 결합해 원하는 정보 얻는 방법

2개 이상 **테이블 연결해 1개 테이블처럼 두고 검색**

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

Right Table

ID	Country
3	Panama
4	Spain

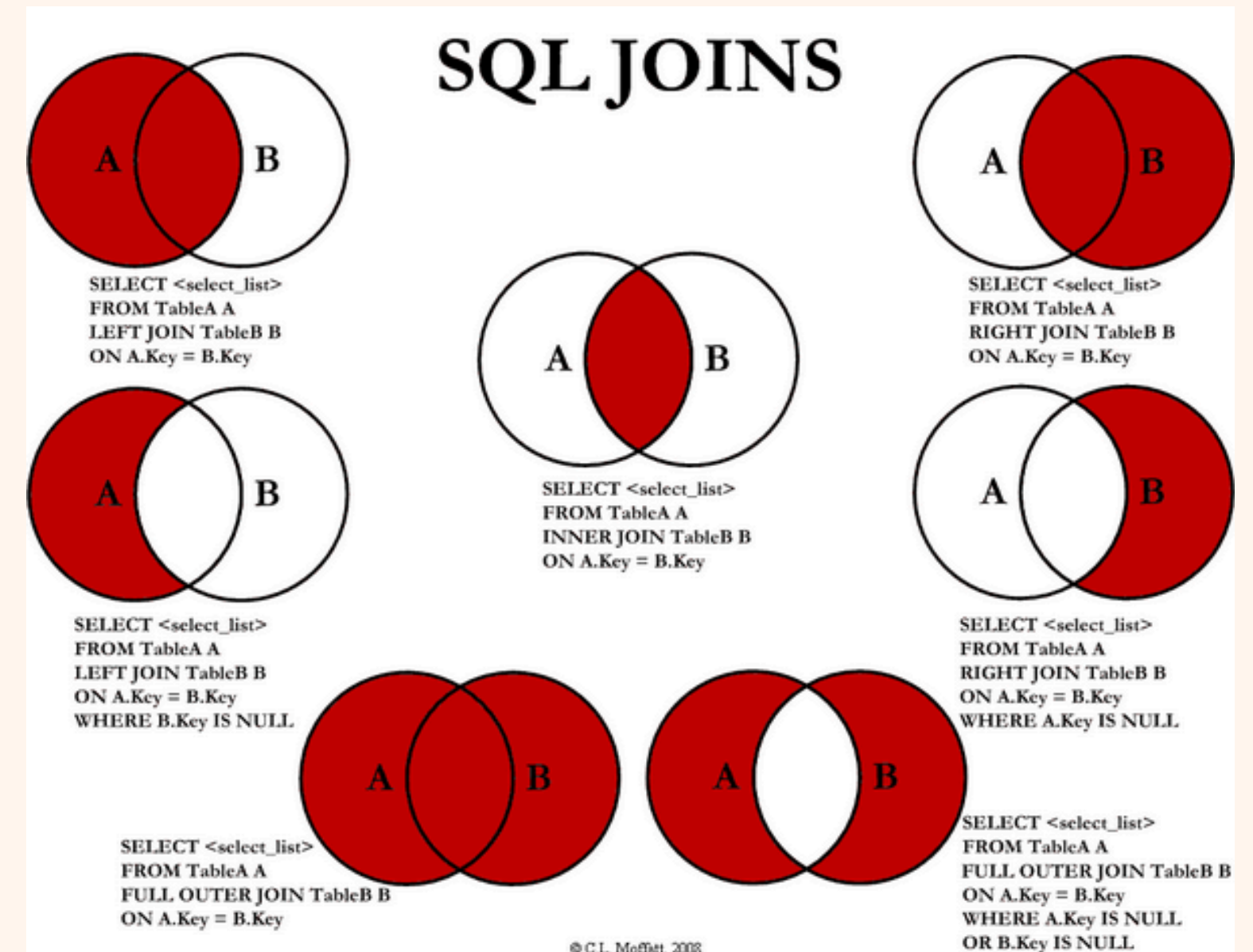


Merged Table

Date	CountryID	Units	Country
1/3/2024	3	30	Panama

Join 종류

- **Inner join**
 - explicit inner join
 - implicit inner join
- **Outer join**
 - left outer join
 - right outer join
 - full outer join
- **cross join**
- **self join**
- **natural join**



목차

1

Join 개념과 종류

2

USING vs ON

3

WHERE vs ON

4

Join과 성능

5

Join 알고리즘

USING vs ON

USING이란

같은 이름을 가진 컬럼을 기준으로 조인하는 방식
결과 테이블에도 해당 컬럼이 **한 번만 나타남**

```
SELECT *  
FROM employee e  
JOIN salary s  
USING(emp_no);
```

USING vs ON

ON이란

두 테이블 연결할 때 **조인 조건 명시**하는 방식
컬럼명 같거나 다르거나 해 각 테이블의 **컬럼 명시적으로 지정**
결과에 조인 조건에 사용된 컬럼이 **양쪽 테이블 모두 출력 가능**

```
SELECT *  
FROM employee e  
JOIN salary s  
ON e.employee_id = s.emp_no;
```


USING vs ON

- 컬럼명

- **using** : 컬럼명 **같은 때만** 사용 가능
- **on** : 컬럼명 **달라도** 사용 가능

- 결과 컬럼

- **using** : 공통 컬럼 **한번만 표시**
- **on** : 조인에 **사용된 컬럼 모두 표시**

- 추가 조건

- **using** : 추가 조건 붙이기 **어려움**
- **on** : 함수, 연산 등 **자유롭게 확장 가능**

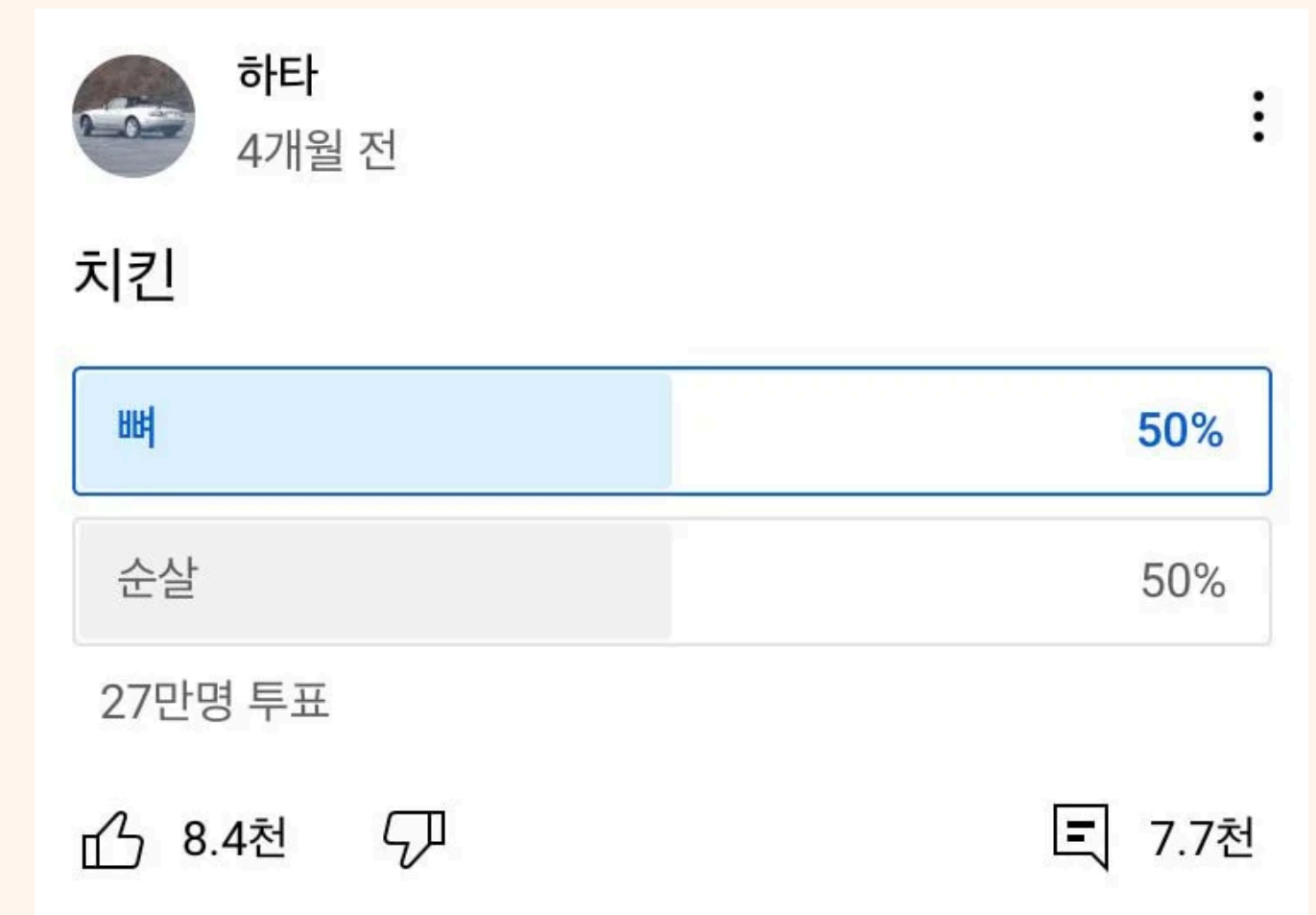
- 가독성

- **using** : 간결, 중복 방지, **가독성 높음**
- **on** : **조건 길어질 수 있음**

USING vs ON

성능 이슈

- 둘 사이 **성능 차이 사실상 없음**
- 컬럼명 같고 단순히 조인 : **USING** 사용
- 컬럼명 다르거나 추가 조건 필요한 경우 : **ON** 사용
- **가독성, 유지 보수성** 중심으로 접근



목차

1

Join 개념과 종류

2

USING vs ON

3

WHERE vs ON

4

Join과 성능

5

Join 알고리즘

WHERE vs ON

ON

- 두 테이블 **결합하는 시점에 조건 검사**해 어떤 행들 조인할지 결정
- join 전에 조건 필터링해 **join할 테이블 줄어듦**
- **조인 대상 행 결정**하는 용도

```
SELECT *  
FROM A  
INNER JOIN B  
ON A.id = B.id AND A.status = 'active';
```

WHERE vs ON

WHERE

- **JOIN 결과 중**에서 **조건 만족**하는 행만 최종 출력
- JOIN 후에 조건 필터링해 **결과 테이블 줄어듦**
- **조인 결과 제한**하는 필터링 용도

```
SELECT *  
FROM A  
INNER JOIN B  
ON A.id = B.id  
WHERE A.status = 'active';
```


WHERE vs ON

JOIN할 테이블

employee 테이블

employee_id	name
1	Alice
2	Bob
3	Carol
4	David

salary 테이블

emp_no	amount
1	5000
2	6000
3	7000

WHERE vs ON

LEFT OUTER JOIN - ON 절에 조건

```
SELECT *  
FROM employee e  
LEFT JOIN salary s  
  ON e.employee_id = s.emp_no AND s.amount > 6000;
```

WHERE vs ON

LEFT OUTER JOIN - ON 절에 조건

JOIN할 때부터 **amount > 6000인 salary**만 employee와 연결

employee 테이블

employee_id	name
1	Alice
2	Bob
3	Carol
4	David

salary 테이블

emp_no	amount
3	7000

salary 테이블
이렇게 변화 후 JOIN

salary 테이블

emp_no	amount
1	5000
2	6000
3	7000

WHERE vs ON

LEFT OUTER JOIN - ON 절에 조건

결과:

employee_id	name	emp_no	amount
1	Alice	NULL	NULL
2	Bob	NULL	NULL
3	Carol	3	7000
4	David	NULL	NULL

WHERE vs ON

LEFT OUTER JOIN - WHERE 절에 조건

```
SELECT *  
FROM employee e  
LEFT JOIN salary s  
    ON e.employee_id = s.emp_no  
WHERE s.amount > 6000;
```

WHERE vs ON

LEFT OUTER JOIN - WHERE 절에 조건

JOIN 후 결과에서 salary amount > 6000인 행만 남김 (NOT NULL만 남음)

JOIN한 직후 결과

employee_id	name	emp_no	amount
1	Alice	1	5000
2	Bob	2	6000
3	Carol	3	7000
4	David	NULL	NULL

WHERE vs ON

LEFT OUTER JOIN - WHERE 절에 조건

JOIN한 직후 결과

employee_id	name	emp_no	amount
1	Alice	1	5000
2	Bob	2	6000
3	Carol	3	7000
4	David	NULL	NULL

결과:

employee_id	name	emp_no	amount
3	Carol	3	7000

WHERE vs ON

성능 이슈 (OUTER JOIN에서)

ON

- 조인 대상 되는 데이터 양 줄어듦
- 조인하는 데이터 작아져 이후 연산 비용 감소

WHERE

- 많은 행 조인돼 일시적으로 더 큰 결과 만들어짐
- 큰 결과에서 다시 필터링



WHERE

ON

목차

1

Join 개념과 종류

2

USING vs ON

3

WHERE vs ON

4

Join과 성능

5

Join 알고리즘

Driving Table 선택

Join에 사용되는 테이블 2개 : Driving Table, Driven Table

Driving Table

- Join시 **먼저 access**하는 테이블
- 2중 for문의 **바깥쪽 for문**

Driven Table

- **Driving table로부터 값 받아**
나중에 access되는 테이블
- 2중 for문의 **안쪽 for문**

Driving Table 선택

동작 방식

- 2개 테이블 중 **Driving, Driven table** 선택
- 각 Driving 테이블 행에 대해 **Driven 테이블 행 순회**하면서 조건에 맞는 행 찾는 것

```
Driving Table 배열 길이 = 500000;  
Driven Table 배열 길이 = 1000;  
  
for(int i=0; i<500000; i++){ <- Driving Table  
    for(int j=0; j<1000; j++){ <- Driven Table  
        if(Driving Table[i]==Driven Table[j]){  
            // Join 수행!  
            break;  
        }  
    }  
}
```


Driving Table 선택

Driving Table

테이블명 : 주문목록

제품번호(FK)	주문번호	주문수량
A01	20090201	5
B01	20090201	3

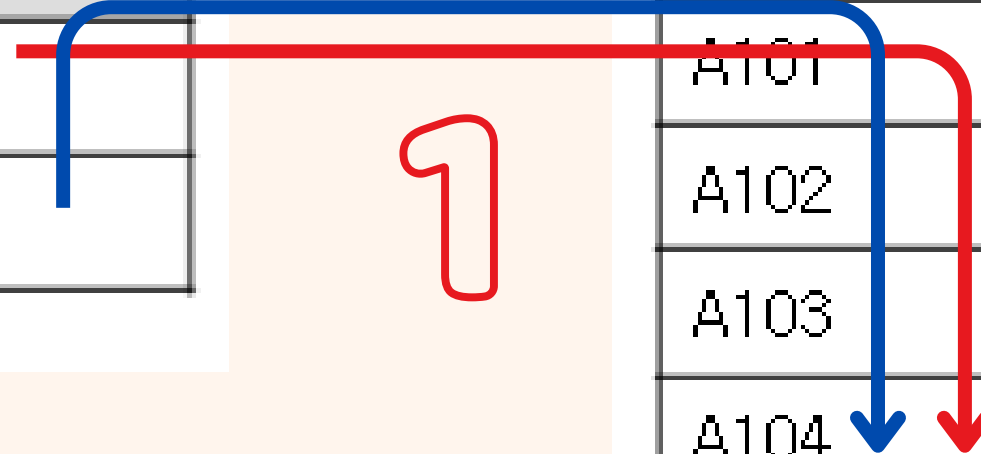
Driven Table

테이블명 : 메뉴

메뉴ID	메뉴명	메뉴기능	상위메뉴ID(FK)
A101	회원관리	회원관리주메뉴	
A102	회원가입	회원가입신청	A101
A103	회원정보수정	회원정보수정	A101
A104	탈퇴신청	회원탈퇴신청	A101

2

1



Driving Table 선택

선택이 성능에 미치는 영향

행 수 적고, 조건으로 걸러질 가능성 높은 테이블 Driving table

- Driving table의 행 수만큼 반복적으로 Driven table 조회

데이터 적은 테이블이 Driving table

- 반복 조회가 적어져 빨라질 수 있음

Driven table join 컬럼에 Index 있을 때 성능 좋음

- Index 있다면 Driven table 순회하지 않고 빨리 찾아 $O(MN) \rightarrow O(M\log N)$

JOIN과 INDEX

JOIN시 INDEX 없다면

전체 테이블 스캔 발생

- JOIN 조건 맞는 **데이터 찾기 위해 전체 테이블 스캔**

JOIN 버퍼 사용

- INDEX 사용 X → JOIN 버퍼 사용해 **메모리 사용량 크게 증가**

JOIN 알고리즘 제한

- INDEX 사용하는 **효율적인 알고리즘 사용 불가**



JOIN과 INDEX

JOIN시 INDEX 있다면

- 조인 조건에 맞는 레코드만 찾아서 **빠른 성능 발휘**
- **전체 테이블 스캔 피해** 성능 향상
- index 활용하는 **효율적인 join 알고리즘 사용 가능**
- **조인 버퍼 사용 X** → 메모리 사용량 낮춤
- **불필요한 연산, IO 작업 감소**해 성능 개선



JOIN과 INDEX

JOIN에서 INDEX 사용 시 주의사항

join 컬럼 데이터 타입 일치 필요

- **타입 다르면** INDEX 활용 X
- **자동 형변환**해 비용 발생

join 대신 대체 수단 고려

- Join 부담될 때 **윈도우 함수 등 고려**

join 조건에 함수, 연산자 주의

- **사용 시 index 무시**됨
- Equi join → INDEX 사용,
- **Not Equi join** → 연산자로 인해 **Index X**

join은 무거운 연산

- join은 **무거운 연산이라 과도하게 남발 X**

목차

1

Join 개념과 종류

2

USING vs ON

3

WHERE vs ON

4

Join과 성능

5

Join 알고리즘

종류 & 동작 방식

Nested Loop Join

- 가장 **기본적이고 단순한** 방식
- Driving table의 각 행 & Driven table의 **모든 행 반복 비교** join 조건 만족하는 데이터 찾을 때까지

Driving Table

테이블명 : 주문목록		
제품번호(FK)	주문번호	주문수량
A01	20090201	5
B01	20090201	3

Driven Table

테이블명 : 메뉴			
메뉴ID	메뉴명	메뉴기능	상위메뉴ID(FK)
A101	회원관리	회원관리주메뉴	
A102	회원가입	회원가입신청	A101
A103	회원정보수정	회원정보수정	A101
A104	탈퇴신청	회원탈퇴신청	A101

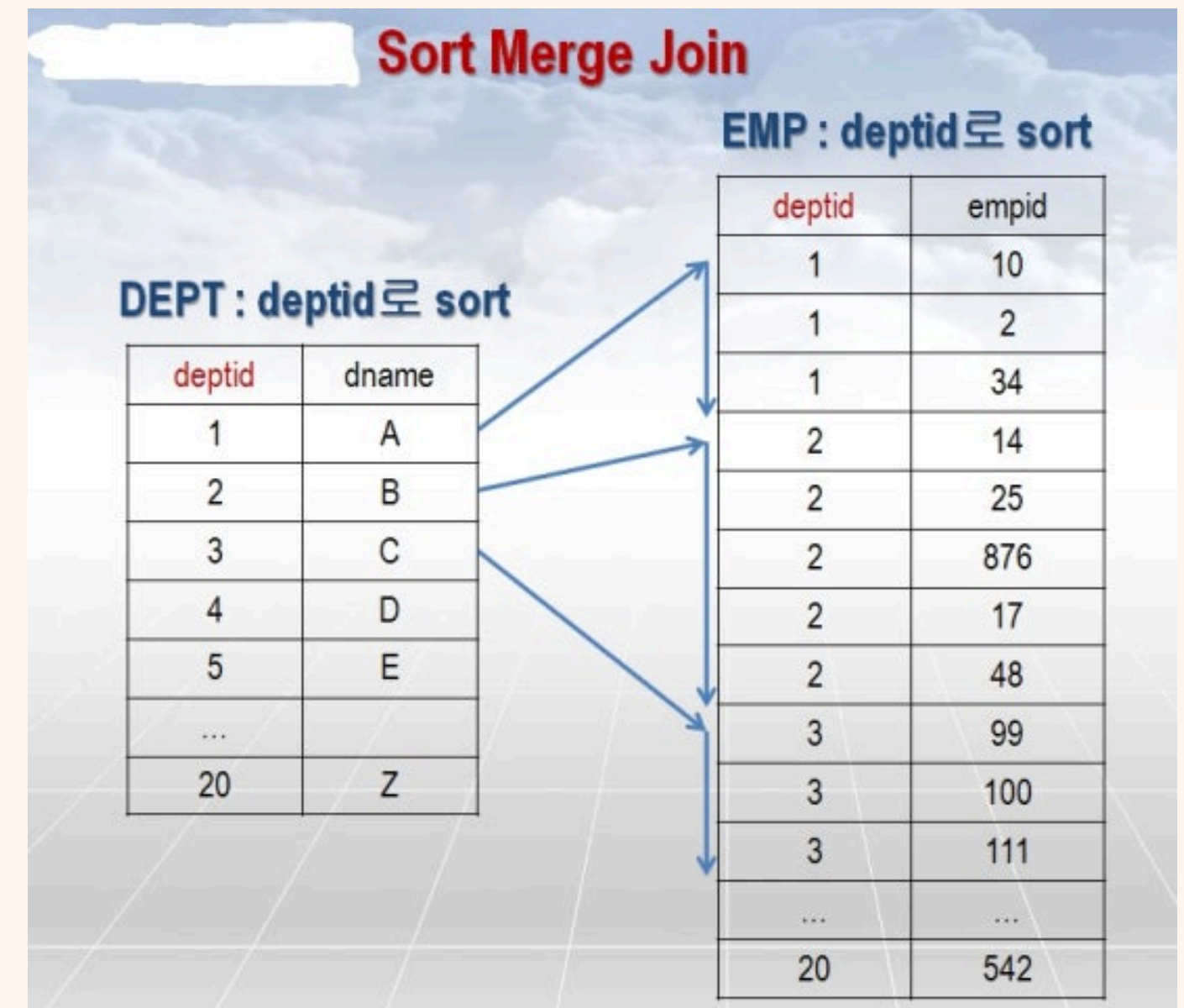
2

1

종류 & 동작 방식

Sort Merge Join

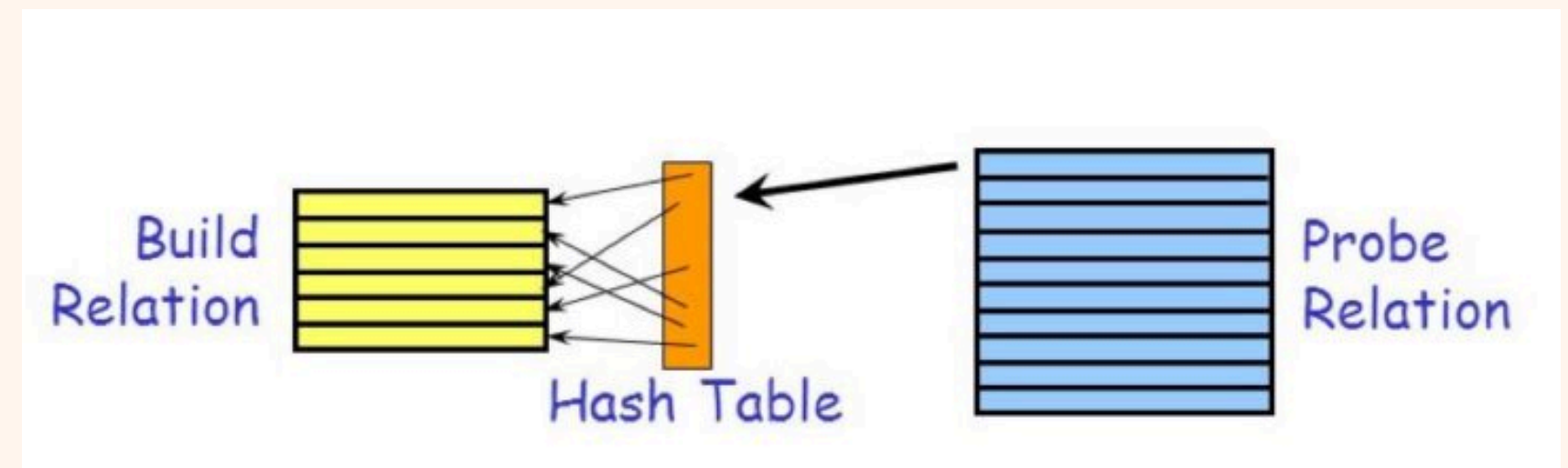
- join 컬럼 기준으로 **두 테이블 먼저 정렬**
- 정렬된 데이터 **양쪽 순차적으로 비교**하며 join 수행



종류 & 동작 방식

Hash Join

- 한 쪽 테이블 해시 테이블로 만듦
- 다른 테이블의 각 행에 해시 함수 적용해 빠르게 매칭되는 데이터 찾아 join



장단점

Nested Loop Join

- 데이터 적을 때 효율적 
- but 데이터 많아질수록 성능 급격 저하 
- 단순 구현이 장점 
- Index 활용 시 성능 향상 가능 

장단점

Sort Merge Join

- 데이터 미리 정렬되어 있으면 매우 효율적 
- 조인에 사용되는 컬럼 index 있으면 매우 효율적 
- 대량의 데이터 처리할 때 좋은 성능 보임 
- 정렬 연산에 따른 추가 비용 발생 가능 

종류 & 동작 방식

Hash Join

- **대량 데이터** 조인에 성능이 좋음 
- **작은 테이블에 적합**함 
- **양쪽 테이블 크기 많이 다를 때** 효율적임 
- **메모리 사용량 많음** 

Quiz

1번 문제

USING과 ON 둘 중

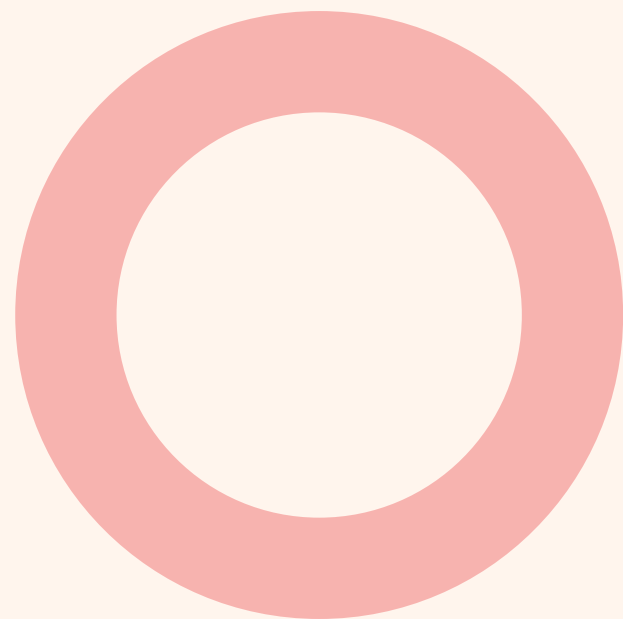
USING는 추가 조건을 붙이기 쉽다



1번 문제

ON이 추가 조건 붙이기 좋음

USING은 컬럼명 같을 때만 사용해 **간결, 가독성 높음**



2번 문제

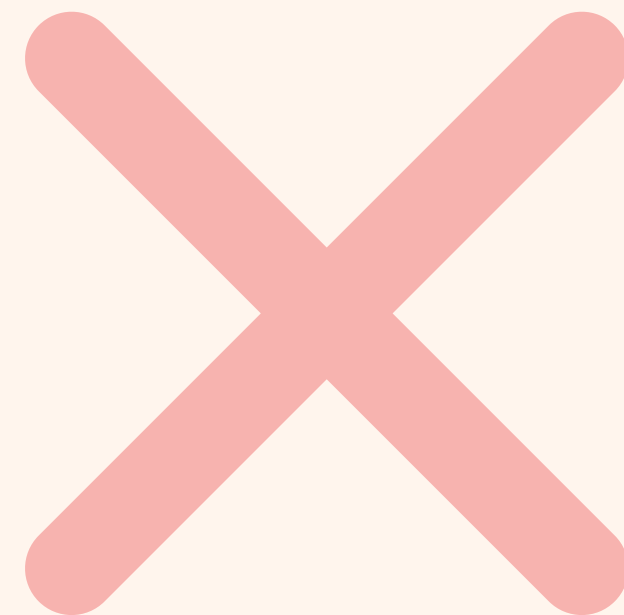
WHERE과 ON 둘 중
ON이 성능이 더 좋다.



2번 문제

ON절이 **join할 테이블의 크기** 자체 줄임

join하는 데이터가 작아져 **연산 비용 감소**



3번 문제

Driving table과 Driven table 중

Driving table에 INDEX가 있는 것이 좋다.



3번 문제

Driven table에 INDEX가 있어야

Driven table **순회하지 않고** 빨리 찾을 수

있어서 **성능 $O(MN)$ 에서 $O(M\log N)$**

